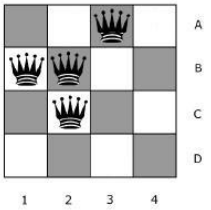
BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Trương Văn Thiện
MSSV / Lớp	3123410356/DCT1232

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a). Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Dựa vào Hình 1, trạng thái khởi đầu của các quân hậu là:

- Quân hậu 1 (Hàng A): Cột 2
- Quân hậu 2 (Hàng B): Cột 4
- Quân hậu 3 (Hàng C): Cột 1
- Quân hậu 4 (Hàng D): Cột 3
- Trạng thái: (2, 4, 1, 3).

Ta kiểm tra số xung đột của từng quân hậu:

- Q1(1,2): Xung đột với Q3(3,1) [đường chéo] và Q4(4,3) [đường chéo] → 2 xung đột.
- Q2(2,4): Xung đột với Q4(4,3) [đường chéo] → 1 xung đột.
- Q3(3,1): Xung đột với Q1(1,2) [đường chéo] → 1 xung đột.
- Q4(4,3): Xung đột với Q1(1,2) [đường chéo] và Q2(2,4) [đường chéo] → 2 xung đột.

Tổng số xung đột hiện tại là 6. Thuật toán Hill Climbing sẽ tìm nước đi di chuyển một quân hậu sao cho số xung đột mới là ít nhất.

Xét các nước đi hợp lệ (di chuyển quân hậu sang cột khác trong cùng hàng):

- Nếu di chuyển Q1 sang cột 1: (1,1) → Xung đột với Q2, Q3, Q4 → 3 xung đột.
- Nếu di chuyển Q2 sang cột 1: (2,1) → Xung đột với Q1, Q3, Q4 → 3 xung đột.
- Nếu di chuyển Q3 sang cột 2: (3,2) → Xung đột với Q1, Q2, Q4 → 3 xung đột.
- Nếu di chuyển Q4 sang cột 1: (4,1) → Xung đột với Q1, Q2, Q3 → 3 xung đột.

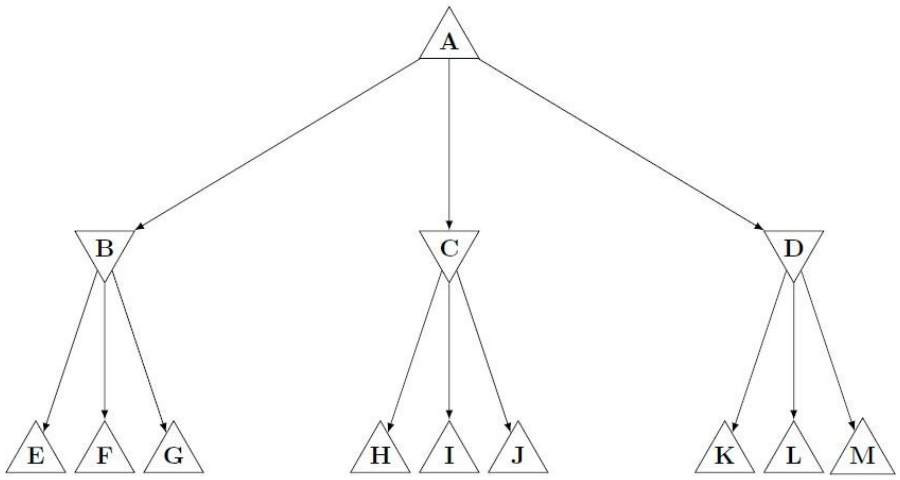
Nhận thấy tất cả các nước đi đơn lẻ đều dẫn đến 3 xung đột (tệ hơn hoặc bằng trạng thái hiện tại). Do đó, thuật toán sẽ không thực hiện di chuyển nào và trả về trạng thái hiện tại. Trạng thái sau một vòng lặp: (2, 4, 1, 3) (Không thay đổi).

b). Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

- Có**,** thuật toán dừng lại ngay lập tức vì trạng thái khởi đầu đã là một giải pháp hợp lệ (0 xung đột) hoặc là một cực tiểu địa phương (không có nước đi nào tốt hơn).
- Trạng thái cuối cùng thuộc loại: C) Cực trị toàn cục (global optimum) (và cũng là B) Cực trị cục bộ (local optimum) vì nó không thể cải thiện).

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

☑ D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

- **Duyệt D (MIN):**
  - Con K = 23 → D = min(23) = 23.
  - Con L = 20 → D = min(23, 20) = 20.
  - Con M = 26 → D = min(20, 26) = 20.
  - Nút D trả về 20. Gốc A (MAX) cập nhật alpha = 20.
- **Duyệt C (MIN):**
  - Con H = 17 → C = min(17) = 17.
  - Vì 17 < alpha (20), ta có thể cắt tỉa (prune) các con còn lại của C.
  - Các nút không được thăm: I, J.
- **Duyệt B (MIN):**
  - Con E = 5 → B = min(5) = 5.
  - Vì 5 < alpha (20), ta có thể cắt tỉa các con còn lại của B.
  - Các nút không được thăm: F, G.

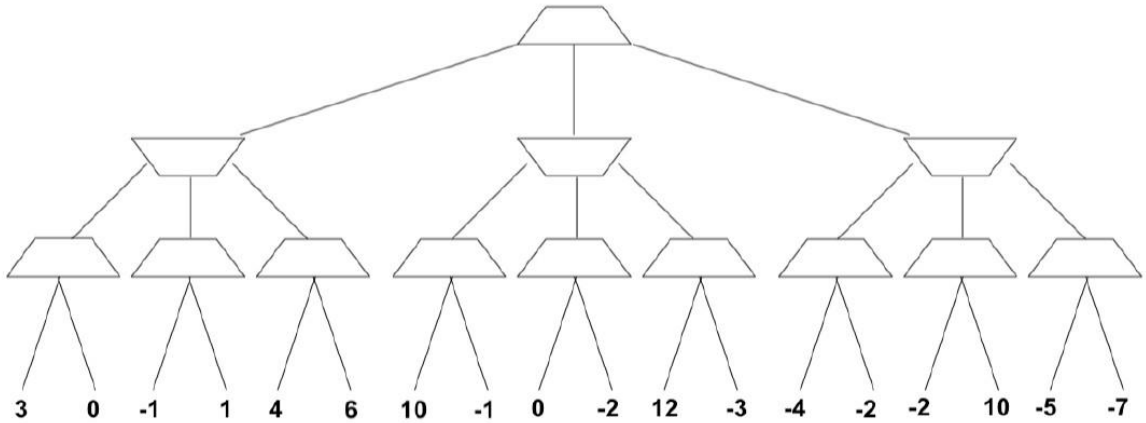
Danh sách các nút không được thăm: I, J, F, G**.**

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

- Alpha-Beta Pruning tiêu chuẩn: Có đảm bảo cùng nước đi với Minimax đầy đủ (với điều kiện thứ tự duyệt giống nhau).
- Iterative Deepening kết hợp Pruning: Không. Vì Iterative Deepening giới hạn độ sâu tìm kiếm, giá trị đánh giá tại các nút bị cắt ngắn có thể khác so với tìm kiếm toàn bộ cây. Điều này có thể dẫn đến việc chọn nước đi tối ưu cục bộ thay vì tối ưu toàn cục.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

- Gốc (MAX), các nút con là MIN, các nút cháu là MAX.
- Giá trị các lá từ trái sang phải: 3, 0, -1, 1, 4, 6, 10, -1, 0, -2, 12, -3, -4, -2, -2, 10, -5, -7.

Quá trình duyệt và cắt tỉa:

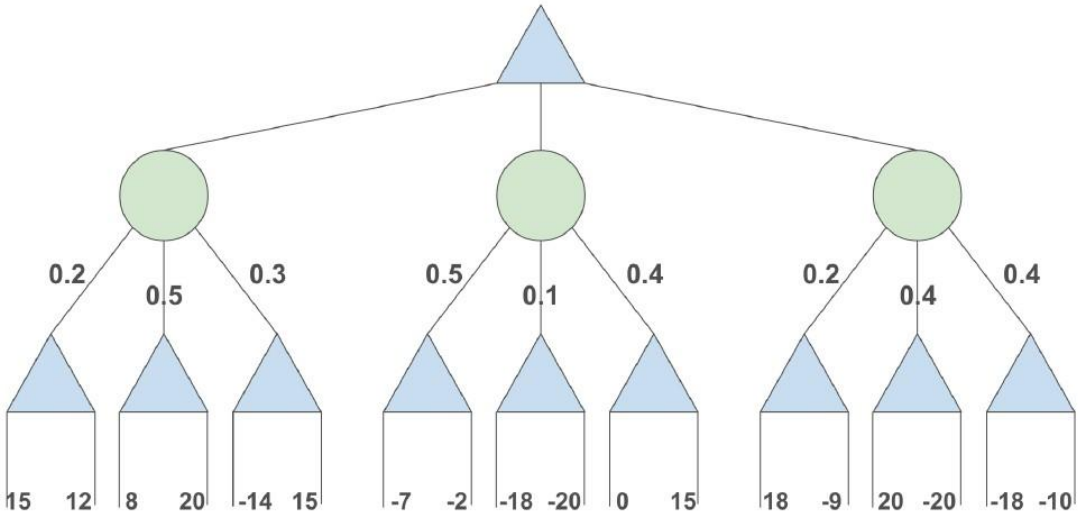
- Nút con 1 (MIN): alpha = -∞, beta = +∞
  - Cháu 1 *(MAX):* Lá 3 → alpha = 3. Lá 0 → alpha = 3. Lá -1 → alpha = 3. Trả về 3.
  - Nút con 1 (MIN)*:* beta = min (+∞, 3) = 3.
  - Cháu 2 (MAX): alpha = -∞, beta = 3. Lá 1 → alpha = 1. Lá 4 → alpha = 4. Vì 4 >= beta (3), **cắt tỉa** lá 6. Trả về 4.
  - Nút con 1 trả về: min (3, 4) = 3.
- **Gốc (MAX):** alpha = max (-∞, 3) = 3.
- **Nút con 2 (MIN):** alpha = 3, beta = +∞
  - Cháu 3 (MAX): Lá 10 → alpha = 10. Lá -1 → alpha = 10. Lá 0 → alpha = 10. Trả về 10.
  - Nút con 2 (MIN): beta = min (+∞, 10) = 10.
  - Cháu 4 (MAX): alpha = 3, beta = 10. Lá -2 → alpha = max (3, -2) = 3. Lá 12 → alpha = max (3, 12) = 12. Vì 12 >= beta (10), cắt tỉa lá -3. Trả về 12.
  - Nút con 2 trả về: min (10, 12) = 10.
- **Gốc (MAX):** alpha = max (3, 10) = 10.
- **Nút con 3 (MIN):** alpha = 10, beta = +∞
  - Cháu 5 (MAX): Lá -4 → alpha = 10. Lá -2 → alpha = 10. Lá -2 → alpha = 10. Trả về 10.
  - Nút con 3 (MIN): beta = min(+∞, 10) = 10.
  - Cháu 6 (MAX): alpha = 10, beta = 10. Lá 10 → alpha = max(10, 10) = 10. Lá -5 → alpha = 10. Lá -7 → alpha = 10. Trả về 10.
  - Nút con 3 trả về: min(10, 10) = 10.

Kết quả:

- Giá trị Minimax tại gốc: 10.
- Nước đi tối ưu của MAX: Chọn nút con 2 (hoặc nút con 3, vì cả hai đều cho giá trị 10).
- Các nhánh bị cắt tỉa: Cạnh từ Cháu 2 đến lá 6, và cạnh từ Cháu 4 đến lá -3. (Đánh dấu X lên các cạnh này).

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

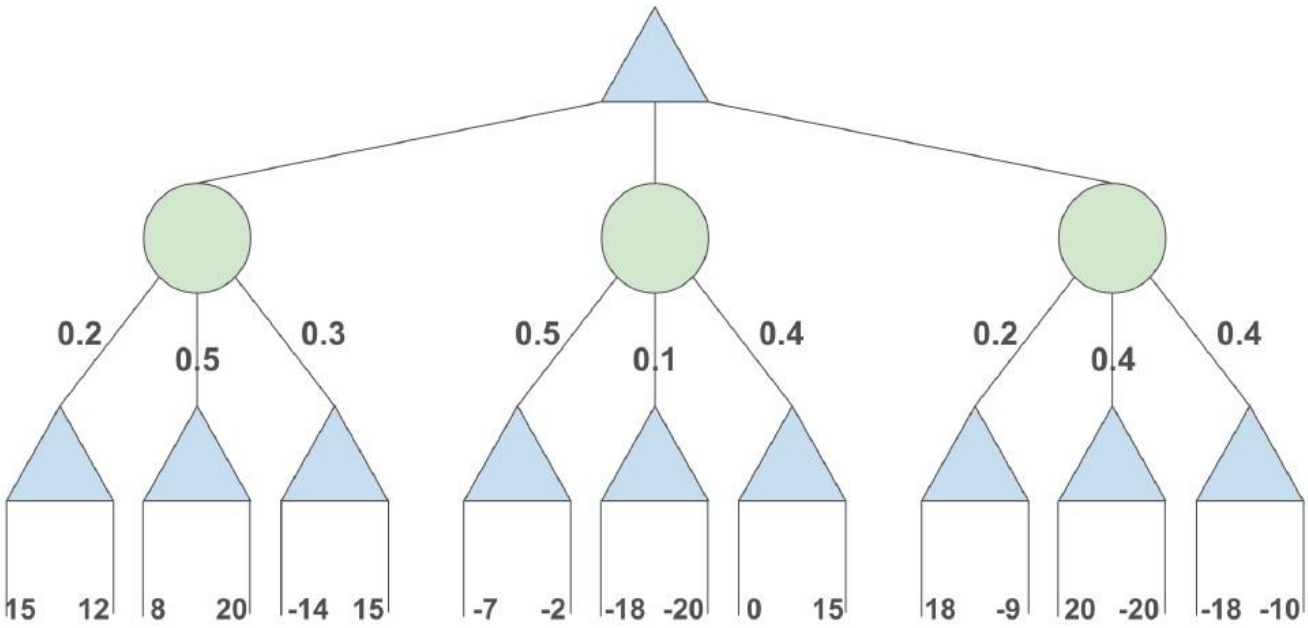
- **Chance 1:** (Xác suất: 0.2, 0.5, 0.3)
  - Nhánh 1 (0.2): MAX (15, 12, 8) = 15
  - Nhánh 2 (0.5): MAX (20, -14, 15) = 20
  - Nhánh 3 (0.3): MAX (-7, -2, -18) = -2
  - Giá trị = 0.2*15 + 0.5*20 + 0.3*(-2) = 3 + 10 - 0.6 = 12.4
- **Chance 2:** (Xác suất: 0.5, 0.1, 0.4)
  - Nhánh 1 (0.5): MAX (-20, 0, 15) = 15
  - Nhánh 2 (0.1): MAX (18, -9, 20) = 20
  - Nhánh 3 (0.4): MAX (-20, -18, -10) = -10
  - Giá trị = 0.5*15 + 0.1*20 + 0.4*(-10) = 7.5 + 2 - 4 = 5.5
- **Chance 3:** (Xác suất: 0.2, 0.4, 0.4)
  - Nhánh 1 (0.2): MAX (18, -9, 20) = 20
  - Nhánh 2 (0.4): MAX (-20, -18, -10) = -10
  - Nhánh 3 (0.4): MAX (15, 12, 8) = 15
  - Giá trị = 0.2*20 + 0.4*(-10) + 0.4*15 = 4 - 4 + 6 = 6

Gốc (MAX): max (12.4, 5.5, 6) = 12.4

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Không thể cắt tỉa bất kỳ nút nào nếu không có giới hạn giá trị của các lá. Trong Expectimax, giá trị của một nút là trung bình có trọng số. Nếu không biết giá trị tối đa có thể có của các lá còn lại, ta không thể chắc chắn rằng nhánh đó sẽ không vượt qua giá trị alpha hiện tại.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

- **Chance 1:** Giá trị = 12.4 (như câu a). Gốc alpha = 12.4.
- **Chance 2:**
  - Nhánh 1 (0.5): MAX = 15.
  - Nhánh 2 (0.1): MAX = 20.
  - Nhánh 3 (0.4): MAX = -10.
  - Giá trị tối đa có thể có của Chance 2 nếu các lá còn lại đều là 20: 0.5*15 + 0.1*20 + 0.4*20 = 7.5 + 2 + 8 = 17.5 > 12.4. → Không thể cắt tỉa.
  - Giá trị thực tế của Chance 2 = 5.5.
- **Chance 3:**
  - Nhánh 1 (0.2): MAX = 20.
  - Nhánh 2 (0.4): MAX = -10.
  - Giá trị tối đa có thể có của Chance 3 nếu nhánh 3 đạt 20: 0.2*20 + 0.4*(-10) + 0.4*20 = 4 - 4 + 8 = 8.

- Vì 8 < alpha (12.4), ta có thể cắt tỉa nhánh 3 của Chance 3 (nhánh chứa các lá 15, 12, 8).

⇒ Kết luận: Có thể cắt tỉa nhánh thứ 3 của Chance 3.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Phạm vi lan truyền: Forward Checking chỉ lan truyền ràng buộc từ biến vừa được gán đến các biến kề chưa được gán. AC-3 lan truyền ràng buộc trên toàn bộ đồ thị (lan truyền toàn cục).

Thời điểm thực hiện: Forward Checking được thực hiện ngay sau mỗi bước gán biến trong quá trình tìm kiếm. AC-3 thường được thực hiện như bước tiền xử lý (trước khi tìm kiếm) hoặc duy trì trong suốt quá trình tìm kiếm.

Chi phí tính toán: Forward Checking có chi phí thấp hơn AC-3 vì nó chỉ kiểm tra các ràng buộc cục bộ. AC-3 có chi phí cao hơn do phải duyệt qua tất cả các cung trong đồ thị nhiều lần.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV): Chọn biến có ít giá trị hợp lệ nhất trong miền giá trị. Heuristic này giúp phát hiện sớm các nhánh cụt (dead ends), từ đó giảm không gian tìm kiếm và tăng hiệu quả của thuật toán Backtracking.

ii) Least Constraining Value (LCV): Khi chọn giá trị cho một biến, ưu tiên chọn giá trị ít hạn chế nhất đối với các biến lân cận. Heuristic này giúp tăng khả năng tìm thấy giải pháp thành công ở các bước tiếp theo.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

- Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.
- Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.
- Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.

- Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C.
- Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng.

a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

- **Biến:** A, B, C, D, E (đại diện cho 5 máy bay).
- **Miền giá trị:** Mỗi biến nhận một cặp giá trị (khung giờ, đường băng).
  - Khung giờ: {1, 2, 3, 4}
  - Đường băng: {Quốc tế (I), Nội địa (D)}
  - Miền ban đầu: 8 giá trị cho mỗi biến.
- **Các ràng buộc:**
  - Unary:
    - B = (1, I) hoặc (1, D). (B hỏng động cơ, phải hạ cánh khung giờ 1).
    - D ∈ {(3, I), (4, I), (3, D), (4, D)}. (D chỉ đến từ khung giờ 3 trở đi).
    - A ∈ {(1, I), (2, I), (1, D), (2, D)}. (A chỉ chờ tối đa đến khung giờ 2).
  - Binary: Thời gian của D < Thời gian của C. (D phải hạ cánh trước C).
  - Toàn cục (Global): Alldiff cho các cặp (khung giờ, đường băng). Không có hai máy bay nào được đặt cùng khung giờ trên cùng một đường băng.

Bổ sung hai ràng buộc sau:

- A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.
- D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

A, B, C chỉ dùng đường băng Quốc tế (I).

D, E chỉ dùng đường băng Nội địa (D).

- Miền mới:
  - A, B, C: {(1, I), (2, I), (3, I), (4, I)}
  - D, E: {(1, D), (2, D), (3, D), (4, D)}
- Đồ thị ràng buộc:
  - Các nút: A, B, C, D, E.

- Các cạnh biểu thị ràng buộc Alldiff (khác khung giờ) giữa các máy bay trên cùng một đường băng:
  - Cạnh giữa tất cả các cặp trong {A, B, C}.
  - Cạnh giữa D và E.
- Cạnh biểu thị ràng buộc thứ tự: D nối với C (D < C).

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

- Áp dụng ràng buộc Unary:
  - B = {(1, I)}
  - D = {(3, D), (4, D)}
  - A = {(1, I), (2, I)}
  - C = {(1, I), (2, I), (3, I), (4, I)}
  - E = {(1, D), (2, D), (3, D), (4, D)}
- Áp dụng Arc Consistency:
  - B = (1, I) → Loại (1, I) khỏi A và C.
    - A = {(2, I)}
    - C = {(2, I), (3, I), (4, I)}
  - D ∈ {(3, D), (4, D)} và D < C → C phải > 3. Vậy C = {(4, I)}.
  - Nếu C = (4, I) thì D phải < 4. Vậy D = {(3, D)}.
  - E không thể trùng khung giờ với D (vì cùng đường băng D). D = (3, D) → Loại (3, D) khỏi E.
    - E = {(1, D), (2, D), (4, D)}
  - A = (2, I). Không ảnh hưởng thêm.
- Miền cuối cùng sau AC:
  - A: {(2, I)}
  - B: {(1, I)}
  - C: {(4, I)}
  - D: {(3, D)}
  - E: {(1, D), (2, D), (4, D)}

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

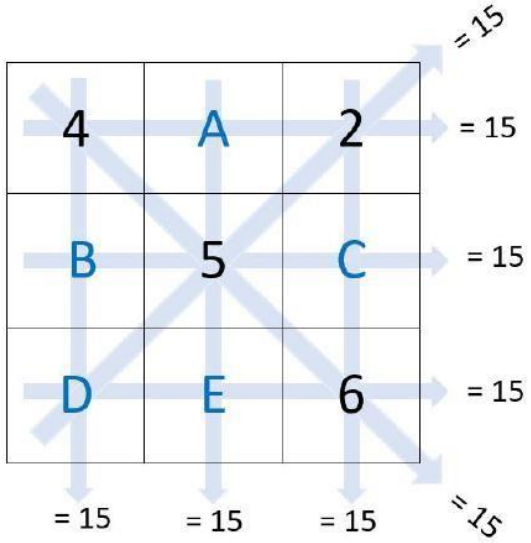
A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

- **Thứ tự chọn biến:** MRV (số giá trị ít nhất), hòa chọn ký tự nhỏ hơn.
- **Thứ tự chọn giá trị:** Giá trị nhỏ hơn trước.
- **Bảng miền ban đầu (sau Unary):**
  - A: {(1, I), (2, I)}
  - B: {(1, I)}
  - C: {(1, I), (2, I), (3, I), (4, I)}
  - D: {(3, D), (4, D)}
  - E: {(1, D), (2, D), (3, D), (4, D)}
- **Quá trình gán:**
  - MRV: B có 1 giá trị. Chọn B = (1, I).
    - Forward Checking: Loại (1, I) khỏi A, C.
    - Miền: A: {(2, I)}, C: {(2, I), (3, I), (4, I)}.
  - MRV: A có 1 giá trị. Chọn A = (2, I).
    - Forward Checking: Loại (2, I) khỏi C.
    - Miền: C: {(3, I), (4, I)}.
  - MRV**:** D và C đều có 2 giá trị. Hòa → chọn C (ký tự nhỏ hơn).
    - Chọn C = (3, I).
    - Forward Checking: D phải < C (3). Nhưng D ∈ {(3, D), (4, D)}. Không có giá trị nào < 3. Dead end Hoàn tác (Backtrack) C.
  - Chọn C = (4, I).
    - Forward Checking: D < 4 → D = {(3, D)}.
    - Miền: D: {(3, D)}.
  - MRV: D có 1 giá trị. Chọn D = (3, D).
    - Forward Checking: Loại (3, D) khỏi E.
    - Miền: E: {(1, D), (2, D), (4, D)}.

- MRV: E có 3 giá trị. Chọn E = (1, D).
    - Miền: E: {(1, D)}.
- Thứ tự gán: (B, (1, I)) → (A, (2, I)) → (C, (3, I)) [Backtrack] → (C, (4, I)) → (D, (3, D)) → (E, (1, D)).
- Số lần backtrack: 1 lần.

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

- Biến: A, B, C, D, E (các ô chưa biết).
- Miền giá trị: {1, 2, 3, 4, 5, 6, 7, 8, 9}. Tuy nhiên, do các số 4, 2, 5, 6 đã được sử dụng, miền thực tế là {1, 3, 7, 8, 9}.
- Ràng buộc:
    - Alldiff(A, B, C, D, E) và khác với các số đã cho.
    - Tổng các hàng, cột, chéo đều bằng 15:
        - Hàng 1: 4 + A + 2 = 15 → A = 9
        - Hàng 2: B + 5 + C = 15
        - Hàng 3: D + E + 6 = 15
        - Cột 1: 4 + B + D = 15
        - Cột 2: A + 5 + E = 15

- Cột 3: 2 + C + 6 = 15 → C = 7
- Chéo 1: 4 + 5 + 6 = 15 (đúng)
- Chéo 2: 2 + 5 + D = 15 → D = 8

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.

- Các nút: A, B, C, D, E.
- Các cạnh nối các biến xuất hiện trong cùng một tổng:
  - A nối với E (Cột 2).
  - B nối với C (Hàng 2), B nối với D (Cột 1).
  - D nối với E (Hàng 3).
- Đồ thị: A-E, B-C, B-D, D-E.

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

- Từ các ràng buộc trên, ta có hệ phương trình:
  - A = 9
  - C = 7
  - D = 8
  - A + 5 + E = 15 → 9 + 5 + E = 15 → E = 1
  - B + 5 + C = 15 → B + 5 + 7 = 15 → B = 3
- Kiểm tra lại: 4 + 3 + 8 = 15 (Cột 1), 8 + 1 + 6 = 15 (Hàng 3). Tất cả đều thỏa mãn.
- Nghiệm: A = 9, B = 3, C = 7, D = 8, E = 1.

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

- AJ = {Đỏ, Vàng, Xanh lá}
- Sunny = {Đỏ, Vàng, Hồng}
- Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

- MRV: Ellyse có 2 giá trị (Đỏ, Hồng), AJ và Sunny có 3 giá trị. Chọn Ellyse.
- LCV: Ellyse chọn giá trị ít hạn chế nhất.
  - Nếu Ellyse = Đỏ: AJ còn {Vàng, Xanh lá}, Sunny còn {Vàng, Hồng}.
  - Nếu Ellyse = Hồng: AJ còn {Đỏ, Vàng, Xanh lá}, Sunny còn {Đỏ, Vàng}.

    - Chọn Ellyse = Hồng (ít hạn chế hơn).
- Tiếp theo: AJ và Sunny đều có 3 giá trị. Hòa → chọn AJ (thứ tự bảng chữ cái).
    - AJ = Đỏ (đầu tiên trong thứ tự cầu vồng: Đỏ, Vàng, Xanh lá).
    - Sunny còn {Vàng} → Sunny = Vàng.
- Thứ tự gán: Ellyse = Hồng → AJ = Đỏ → Sunny = Vàng.
- Số lần backtrack: 0.

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

- Maximum Remaining Values: Chọn AJ hoặc Sunny (3 giá trị). Hòa → chọn AJ.
- Most Constraining Value**:** AJ chọn giá trị hạn chế nhất.
    - AJ = Đỏ: hạn chế Sunny (mất Đỏ) và Ellyse (mất Đỏ).
    - AJ = Vàng: hạn chế Sunny (mất Vàng).
    - AJ = Xanh lá: không hạn chế ai.
    - Chọn AJ = Đỏ.
- Tiếp theo: Sunny (2 giá trị: Vàng, Hồng) và Ellyse (1 giá trị: Hồng). Maximum Remaining Values → chọn Sunny.
    - Sunny = Hồng (hạn chế Ellyse) hoặc Vàng (không hạn chế). Chọn Sunny = Hồng.
    - Ellyse = Đỏ.
- Thứ tự gán: AJ = Đỏ → Sunny = Hồng → Ellyse = Đỏ.
- Số lần backtrack: 0.
- So sánh: Cả hai cách đều không cần backtrack trong trường hợp này. Tuy nhiên, Maximum Remaining Values và Most Constraining Value thường kém hiệu quả hơn vì chúng không ưu tiên các biến bị hạn chế nhất, dẫn đến việc phát hiện dead end muộn hơn. Trong bài toán cụ thể này, cả hai đều may mắn tìm thấy giải pháp ngay lập tức.

--- Hết ---